

Mise en production d'un modèle de prédiction des loyers au Sénégal

Rapport technique court - architecture, expérimentation, validation et limites

Livrable	État vérifié
Données	1 200 annonces synthétiques reproductibles
Modèles	Ridge et Random Forest suivis avec MLflow
Qualité	11 tests, couverture 82,81 %, mypy strict et ruff
Service	FastAPI + Streamlit + Docker Compose

Version de soutenance - 27 juillet 2026

1. Données et préparation

Nous avons retenu l'option des données synthétiques. Ce choix rend le projet reproductible, évite les risques juridiques du scraping et permet de documenter exactement la logique de génération. Il ne prétend pas reproduire le marché réel.

Variables et règles de cohérence

Famille	Variables	Principe
Localisation	ville, quartier	Dakar et quartiers premium plus chers
Bien	type, surface, pièces, chambres	Prix croissant avec surface et capacité
Services	meublé, équipements	Prime mesurée selon le confort
Cible	prix mensuel	FCFA arrondi à 5 000 avec bruit log-normal

Contrôles appliqués

- graine 42 et 1 200 lignes pour la reproductibilité ;
- validation des colonnes obligatoires avant toute transformation ;
- imputation explicite du quartier, des équipements et du statut meublé ;
- rejet des surfaces, prix et nombres de pièces impossibles ;
- séparation raw/processed et notebook sans logique de production.

Risque principal : les relations injectées par le générateur peuvent être apprises plus facilement que celles d'un marché réel. Les métriques mesurent donc la cohérence du pipeline, pas une précision commerciale.

2. Modélisation et tracking MLflow

Les deux modèles utilisent le même split 80/20 et le même pipeline de préparation. Les catégories sont imputées puis encodées en one-hot ; les variables numériques sont imputées puis standardisées. Le pipeline complet est sérialisé afin d'éviter tout écart entre entraînement et inférence.

Modèle	MAE (FCFA)	RMSE (FCFA)	R ²	Décision
Ridge	95 675	136 033	0,843	Baseline interprétable
Random Forest	87 156	129 413	0,858	Retenu

Lecture : la MAE du modèle retenu signifie une erreur absolue moyenne d'environ 87 000 FCFA sur le jeu de test synthétique. Le R² de 0,858 signifie que 85,8 % de la variance du prix de ce jeu est expliquée par le pipeline.

Preuves MLflow

Run	Identifiant MLflow	Lignes train
Ridge	3e2bc5d174e8480bb184243ccf1af4fd	960
Random Forest	5b036f203a28485c9230c7edb88868da	960

MLflow trace les paramètres, les trois métriques et l'artefact joblib de chaque run. La base locale SQLite se consulte avec : `mlflow ui --backend-store-uri sqlite:///mlflow.db`.

3. Architecture de production

Composant	Responsabilité	Protection apportée
Hydra	chemins, seed, modèle, hyperparamètres	aucun réglage métier dispersé
Package src	nettoyage, features, entraînement, prédiction	code testable et réutilisable
FastAPI/Pydantic	contrat HTTP et validation	422 sur entrées invalides
Streamlit	formulaire non technique	gestion succès et erreur
Docker Compose	API + front	démarrage reproductible
GitHub Actions	lint, format, types, tests, build	échec précoce avant image

Flux d'une requête

Formulaire Streamlit → POST /predict → validation Pydantic → construction des mêmes features qu'à l'entraînement → pipeline sérialisé → réponse typée en FCFA. Le modèle est chargé une fois grâce à une dépendance mise en cache, jamais à chaque requête.

Contrats API

- GET /health : disponibilité du service et du modèle ;
- GET /model/info : version, date, métriques, run MLflow et volume d'entraînement ;
- POST /predict : données contraintes (ville enum, surface > 0, bornes de pièces) ;
- règle croisée : le nombre de chambres ne dépasse pas le nombre de pièces.

Choix important : prétraitement et modèle sont sauvegardés ensemble. Sauvegarder uniquement le régresseur créerait un risque de colonnes ou d'encodage incompatibles.

4. Qualité, déploiement et limites

Résultats de validation

Contrôle	Résultat local
ruff	aucune erreur
black	format conforme
mypy --strict src/	aucune erreur
pytest	11 tests réussis
couverture src/	82,81 % (seuil : 70 %)
API réelle	health 200, predict 200, invalide 422

Conteneurisation

Les deux images utilisent un build multi-stage, une base python:3.11-slim, un utilisateur non-root, des variables d'environnement et un healthcheck. Le front attend que l'API soit saine. La commande de démonstration est unique : `docker compose -f docker/docker-compose.yml up --build`.

Limites et suite réaliste

- remplacer progressivement les données synthétiques par des données autorisées et datées ;
- surveiller la dérive des distributions et l'erreur sur des loyers réellement observés ;
- déployer MLflow et les artefacts sur un stockage persistant distant ;
- ajouter authentification, limitation de débit et observabilité ;
- enregistrer la version des données et automatiser le réentraînement seulement après validation.

Conclusion : le projet privilégie une chaîne simple, traçable et testée. C'est cohérent avec le sujet, qui note l'ingénierie MLOps avant la sophistication algorithmique.